1 Discuss the memory hierarchy in computer system.

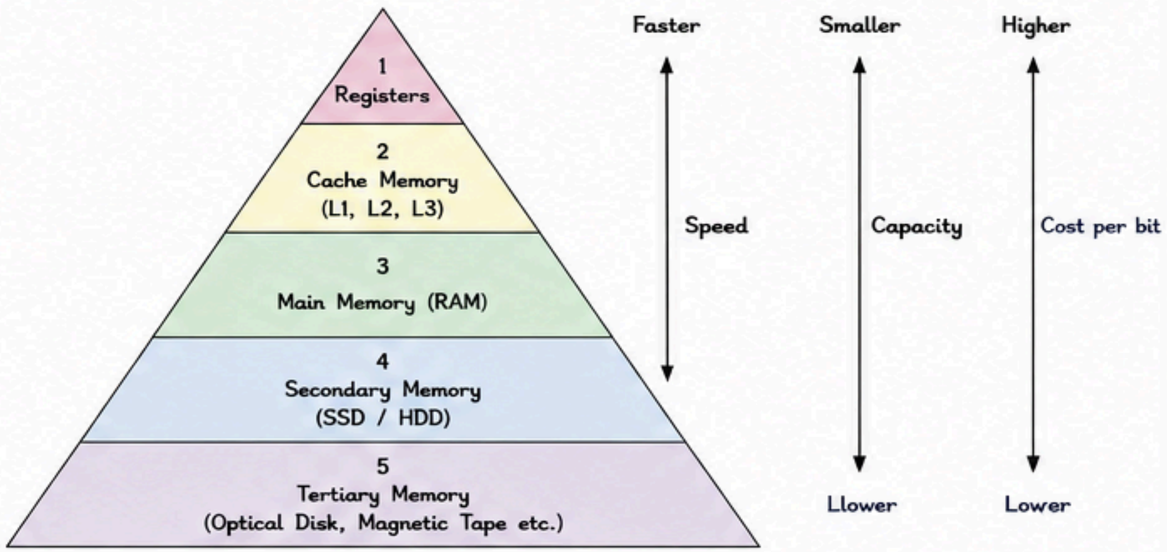
Ans:

Memory hierarchy is the organization of different types of memory in a computer system in a top-down manner based on their speed, capacity and cost.

Characteristics:

- **Speed** : As we move from top to bottom, speed decreases.
- **Capacity** : As we move from top to bottom, capacity increases.
- **Cost** : As we move from top to bottom, cost per bit decreases.
- **Frequency of access** : Top levels are accessed more frequently.
- **Location** : Top levels are usually inside the CPU chip; lower levels are outside the CPU.
- **Management** : Data moves between adjacent levels under the control of hardware or software.

Memory Hierarchy (Top to Bottom)



Explanation of levels:

1. **Registers** : Smallest and fastest storage inside CPU. Hold operands, addresses and intermediate results.
2. **Cache Memory** : Very fast SRAM memory. Stores recently/frequently used data and instructions.
3. **Main Memory (RAM)** : Large and relatively fast memory. Holds programs and data currently in use.
4. **Secondary Memory** : Non-volatile storage with large capacity (e.g., SSD, HDD). Slower than RAM.
5. **Tertiary Memory** : Very large capacity, removable storage (e.g., optical disks, tapes). Slowest.

> **Note:** The objective of memory hierarchy is to present the user with a memory system that is large in capacity, fast in access and low in cost per bit.

2 Explain the hardware organization of associative memory with the help of block diagram.

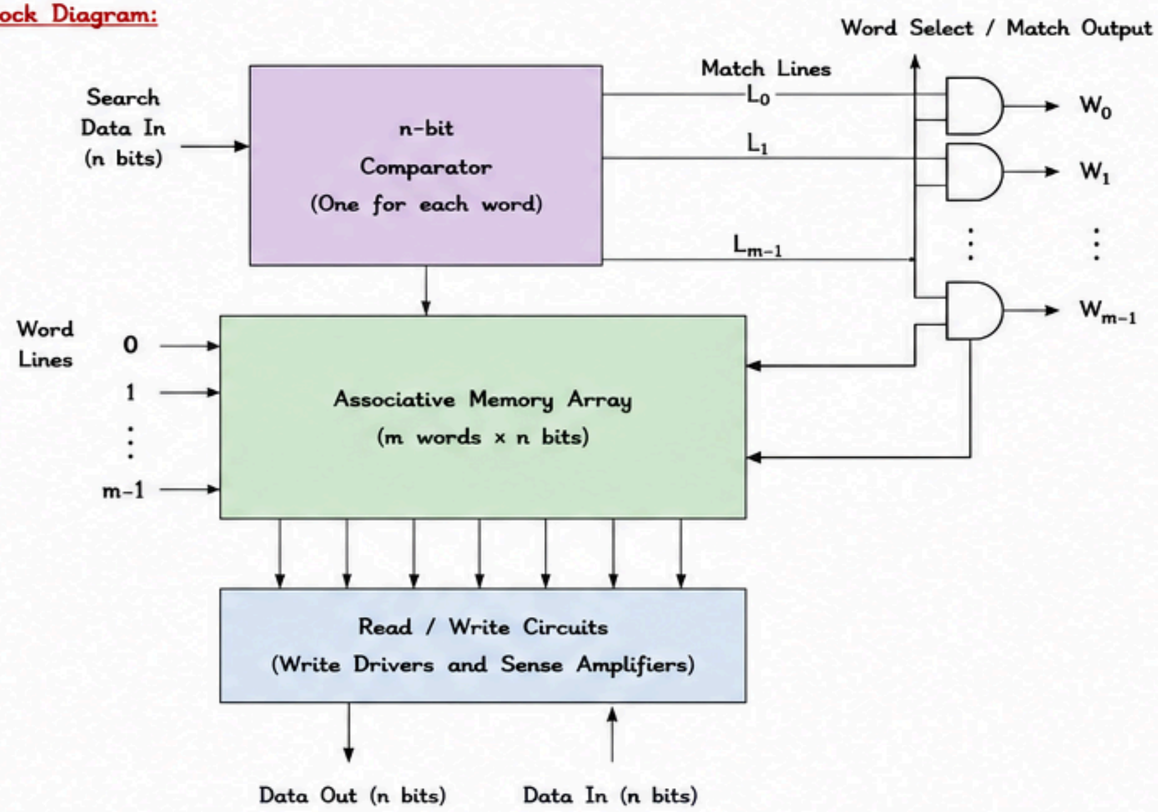
Ans:

Associative memory (Content Addressable Memory) is a special type of memory in which data is accessed by content rather than by address.

Hardware organization:

It consists of a number of words (rows) each of n bits. All words are compared in parallel with the input data. If a match occurs, the corresponding word line is selected and the data can be read or written.

Block Diagram:



Working:

- The search data is applied to all comparators in parallel.
- Each comparator compares the search data with one word in the associative memory array.
- If a match is found (all n bits equal), the corresponding match line becomes 1.
- The match line enables the output gate and the word number is obtained at the output.
- For read operation, the matched word is read out; for write operation, the new data is written into the selected word.

> **Note:** m = number of words, n = number of bits per word.

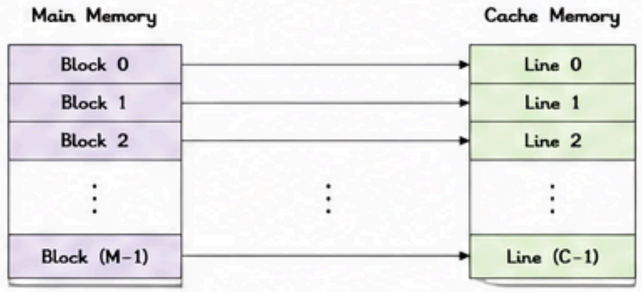
3 Illustrate the mapping techniques in cache memory.

Ans: Mapping is the technique of determining the location in cache where a block from main memory can be placed.

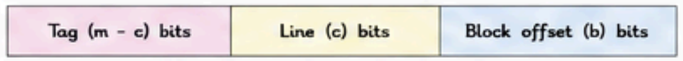
Let, M = 2^{m} = Number of blocks in main memory
C = 2^{c} = Number of lines (blocks) in cache memory
b = Number of block offset bits (Block size = 2^{b} bytes)

(a) Direct Mapping

- Each main memory block can be placed in only one particular cache line.
- Line number = Block number mod C
- Simple and fast.

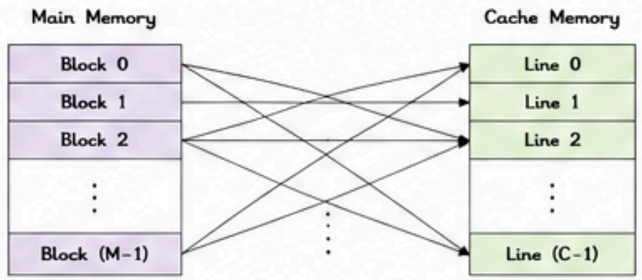


Address format:



(b) Fully Associative Mapping

- A block from main memory can be placed in any cache line.
- Requires comparator for every cache line.
- Maximum flexibility but more hardware cost.

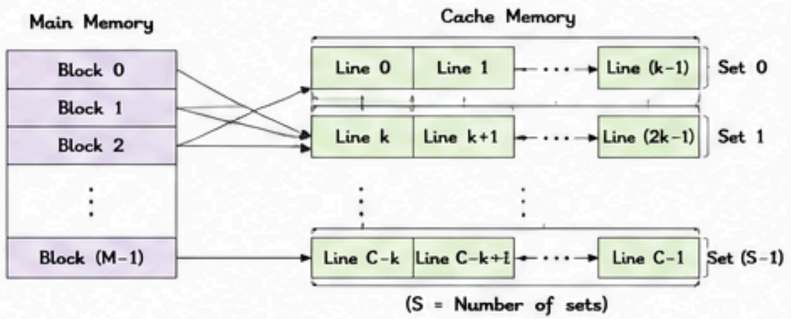


Address format:



(c) Set Associative Mapping (k-way)

- Cache is divided into sets with k lines in each set.
- A block can be placed in any line of its corresponding set.
- k = C / (Number of sets)
- Trade-off between the above two methods.



Address format:



Note: In all the techniques, the block offset (b) bits select a byte/word inside the block. Direct Mapping is simplest, Fully Associative gives highest hit ratio, Set Associative is a compromise between hardware cost and performance.

4 Describe the features of virtual memory and explain the address translation in virtual memory.

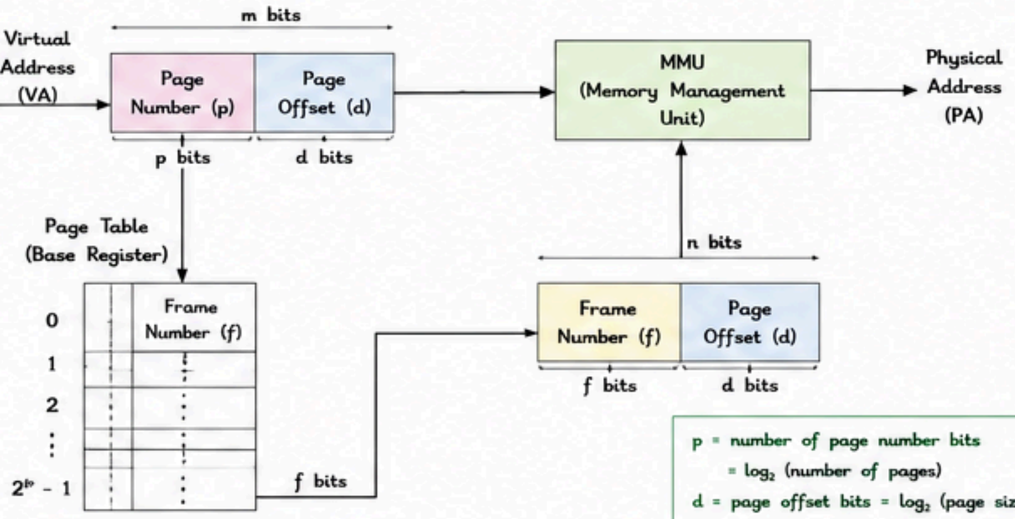
Ans:

(A) Features of Virtual Memory

1. **Large address space:** Provides each process with a large (often 2^{h} byte) logical address space independent of the actual physical memory size.
2. **Protection:** Each process has its own virtual address space; prevents one process from accessing another process's memory.
3. **Privacy:** Keeps code, data and stack of a process separate and hidden from others.
4. **Better memory utilization:** Only the required parts of a program are kept in physical memory. Unused parts remain on secondary storage.
5. **Multiprogramming:** More processes can be kept in main memory simultaneously.
6. **Demand paging:** Pages are brought into memory only when they are referenced (lazy loading).
7. **Easy programming:** Programmer need not worry about actual physical memory size and placement.
8. **Relocation:** Program can be loaded at any location in physical memory.

(B) Address Translation in Virtual Memory

Virtual address (VA) generated by CPU is translated into Physical address (PA) by the MMU using page table.



p = number of page number bits = $\log_2$ (number of pages)
d = page offset bits = $\log_2$ (page size)
f = number of frame bits = $\log_2$ (number of frames)
n = physical address bits = f + d

Working:

1. CPU generates a virtual address (VA) = (p, d) where p is page number and d is offset.
2. The page number (p) is used to index the page table.
3. The page table entry gives the corresponding frame number (f) in physical memory.
4. The physical address (PA) is formed by concatenating frame number (f) and page offset (d), i.e., PA = (f, d).
5. The memory is accessed using the physical address.

Note: If the page table entry is invalid (page not in memory), a page fault occurs and the page is brought from secondary storage into a free frame, then translation is repeated.

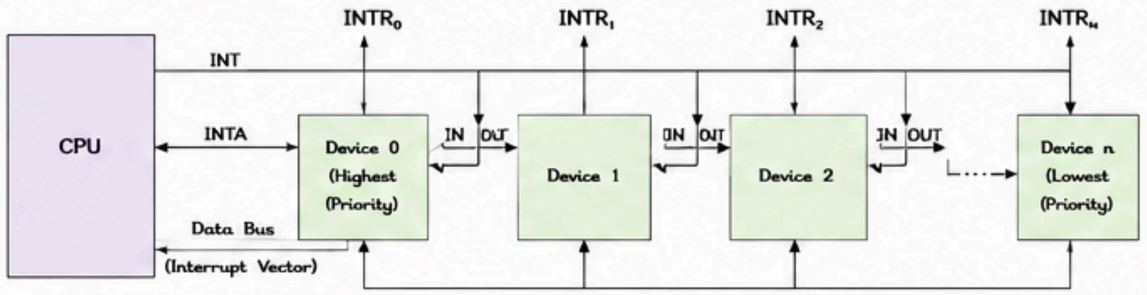
7 Illustrate the "Daisy Chaining method" and "Parallel Priority Interrupt method" of interrupt handling.

Ans:

When multiple devices request interrupt simultaneously, CPU has to decide which interrupt to service first. This is done by different interrupt handling techniques.

(A) Daisy Chaining Method

- The devices are connected in a serial chain.
- An interrupt acknowledge (INTA) signal from CPU is passed from one device to the next.
- The first (highest priority) device that has requested the interrupt captures the INTA signal and places its interrupt vector on the data bus. Lower priority devices do not respond.

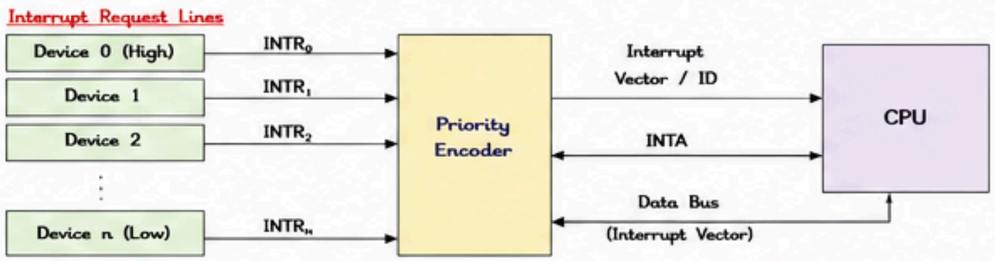


Working:

- All devices assert their interrupt request lines (INTR).
- CPU issues INTA (Interrupt Acknowledge). It goes to Device 0.
- If Device 0 has requested interrupt, it captures INTA, places its vector on data bus and stops the INTA from going further.
- If not, it passes INTA to Device 1; the process continues.
- The first device in the chain that has requested interrupt responds and others are ignored.

(B) Parallel Priority Interrupt Method

- All devices send their interrupt request lines in parallel to a Priority Encoder.
- The encoder identifies the highest priority request and sends interrupt vector (or ID) to CPU.
- CPU issues INTA; the selected device places its vector on data bus.



Working:

- All devices assert their INTR lines simultaneously.
- Priority Encoder gives the highest priority request sends its vector/ID to CPU.
- CPU issues INTA; the selected device puts its vector on data bus.
- Simple, fast and does not suffer from propagation delay as in daisy chain.

Comparison:

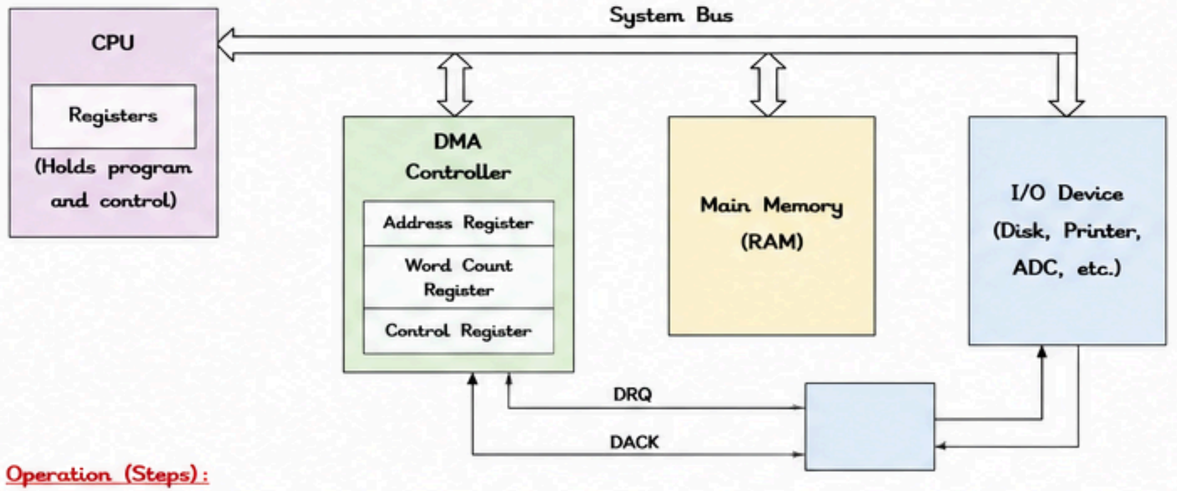
Feature	Daisy Chaining	Parallel Priority
Connection	Serial (chain)	Parallel
Hardware	Simple	More (encoder)
Speed	Slower	Faster
Priority	Fixed by position	By priority encoder

8 Explain the Direct Memory Access (DMA) mode of data transfer with a neat diagram.

Ans:

In DMA mode, a DMA controller transfers data directly between an I/O device and main memory without the CPU's direct involvement in each byte/word transfer.

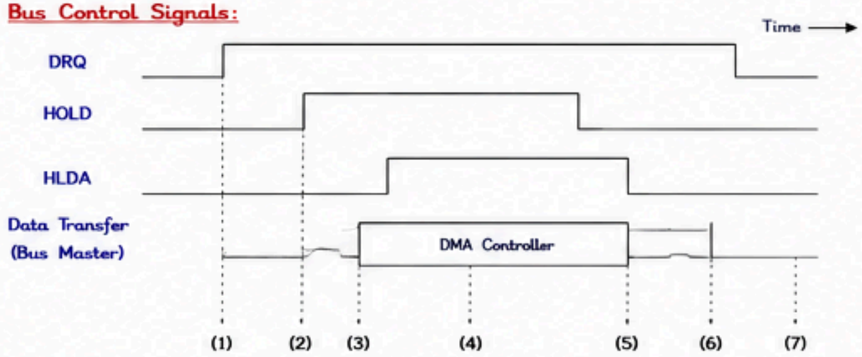
Block Diagram:



Operation (Steps):

1. CPU initializes the DMA controller by writing starting memory address, number of bytes/words to be transferred, direction (I/O → Memory or Memory → I/O) and mode.
2. I/O device requests DMA transfer by asserting DRQ (DMA Request).
3. DMA controller requests control of system bus from CPU by asserting HOLD.
4. CPU, after completing the current bus cycle, responds with HLDA (Hold Acknowledge) and releases the bus.
5. DMA controller becomes bus master and transfers data directly between I/O device and memory. It automatically updates address and count after each transfer.
6. When transfer is complete, DMA controller interrupts the CPU (TC – Terminal Count interrupt).
7. CPU regains the bus and continues the program.

Bus Control Signals:



Key:

1. Device requests DMA.
2. DMA requests bus.
3. CPU acknowledges.
4. DMA transfers data.
5. DMA transfer complete.
6. CPU regains bus.
7. DMA interrupt to CPU.

Advantages:

- High speed data transfer.
- CPU is free to perform other tasks during transfer.
- Efficient for block data transfers.

Applications:

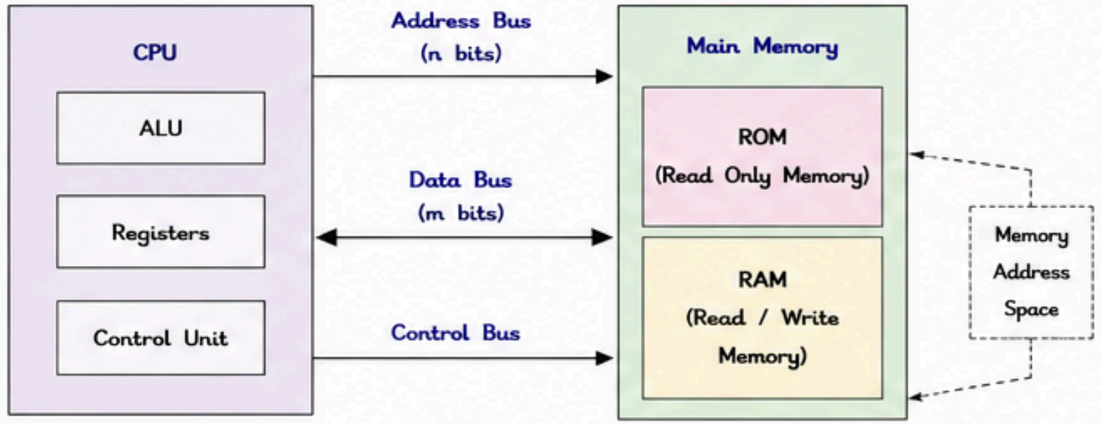
- Disk drives, Printers, Scanners, ADC/DAC, Network devices etc.
- Used in systems where large amount of data is transferred.

5 Explain how the CPU is connected to main memory (RAM and ROM) with a neat diagram.

Ans:

The CPU is connected to main memory (RAM and ROM) through address, data and control buses. The address bus is used to select a memory location, the data bus is used to transfer data and the control bus is used to carry control and timing signals.

Block Diagram :



Buses and Signals :

Bus / Signal	Function	Examples
Address Bus	Carries address from CPU to memory. Selects one location.	A_0 – A_{n-1}
Data Bus	Carries data between CPU and memory. Bidirectional.	D_0 – D_{m-1}
Control Bus	Carries control and timing signals to coordinate operations.	RD (Read), WR (Write), MEMR, MEMW, M/IO, RESET, CLK

Operation :

- To read: CPU places address on address bus, asserts RD signal. Memory places data on data bus which CPU reads.
- To write: CPU places address and data on buses, asserts WR signal. Memory stores the data.
- ROM is generally read-only (connected as read device).
- RAM is read/write (connected for both read and write operations).

Note: The number of address lines (n) determines the maximum memory size.
Memory size = 2^n locations × m bits.

6 Explain the asynchronous data transfer modes.

Ans:

In asynchronous data transfer, data is transferred between two units that do not share a common clock. The timing of transfer is controlled by handshaking signals.

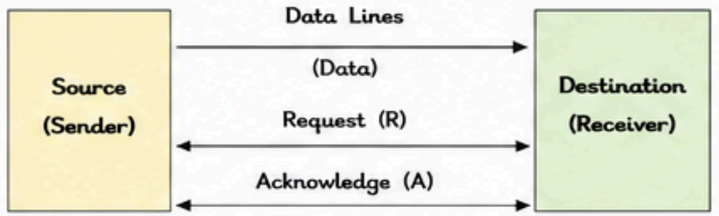
Main Handshaking Signals:

Signal	Meaning
Request (R)	Sender indicates it has data (or is ready).
Acknowledge (A)	Receiver indicates it is ready (or data accepted).

The transfer is completed only after request and acknowledge go through the cycle.

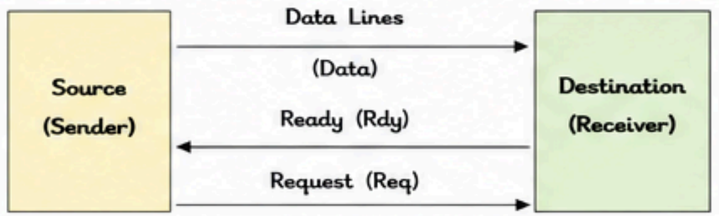
Asynchronous Data Transfer Modes:

1. Source Initiated (Request/Acknowledge) Mode



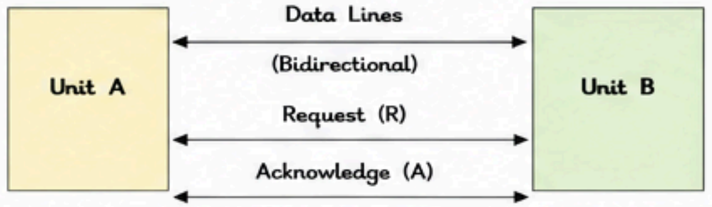
- Source places data on data lines and asserts Request (R).
- Destination, when ready, accepts data and asserts Acknowledge (A).
- Source removes R after receiving A. Transfer complete.

2. Destination Initiated (Ready/Request) Mode



- Destination asserts Ready (Rdy) when it is ready to receive.
- Source places data on data lines and asserts Request (Req).
- Destination accepts data and de-asserts Rdy. Transfer complete.

3. Two-way (Bidirectional) Transfer



- Either unit can act as sender or receiver.
- R and A are used in both directions.

Note:

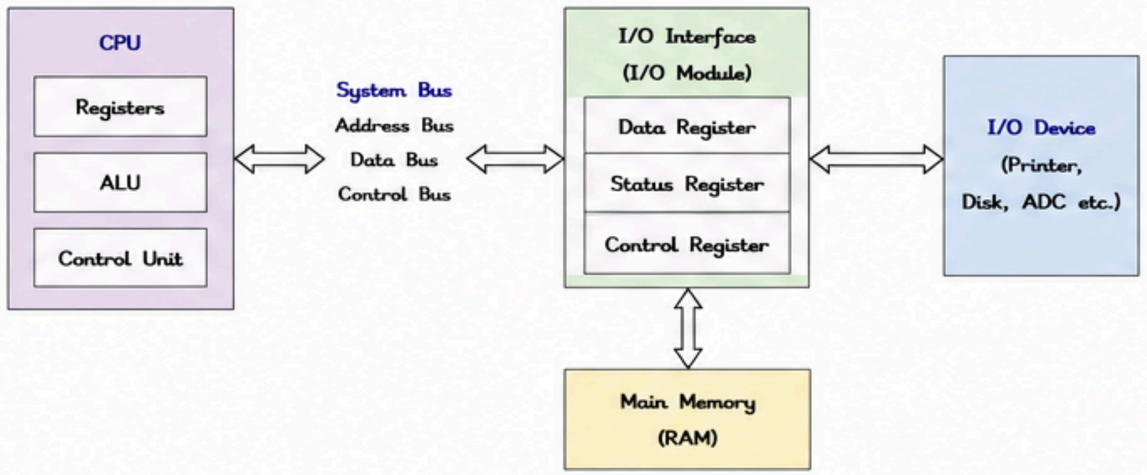
- Asynchronous transfer is slower than synchronous but flexible and simple.
- It is used when data items are infrequent or when two units have different clock speeds.

9 Explain the Programmed I/O mode of data transfer with a neat diagram.

Ans:

In Programmed I/O (also called polling), the CPU transfers data between an I/O device and memory through I/O module. All transfer operations are performed by the CPU under program control and the CPU waits (polls) for the device status.

Block Diagram :



Operation (Read from I/O Device to Memory) :

1. CPU issues a Read command to I/O module by writing appropriate control information.
2. CPU repeatedly reads the status register (polling) until the device reports "Ready".
3. When ready, CPU reads data from data register.
4. CPU stores the data into memory.
5. Steps 2-4 are repeated for all words/bytes.

I/O Interface Registers

Register	Function
Data Register (DR)	Holds data being transferred between CPU and device.
Status Register (SR)	Contains status bits (Ready, Busy, Error, Done, etc.).
Control Register (CR)	Holds control information (Read, Write, Start, Stop, etc.).

Advantages :

- Simple and easy to implement.
- No special hardware like DMA controller.
- Suitable for small/low speed devices.

Limitations :

- CPU is fully occupied during transfer (busy waiting).
- Inefficient for large data transfers.
- Low overall system throughput.

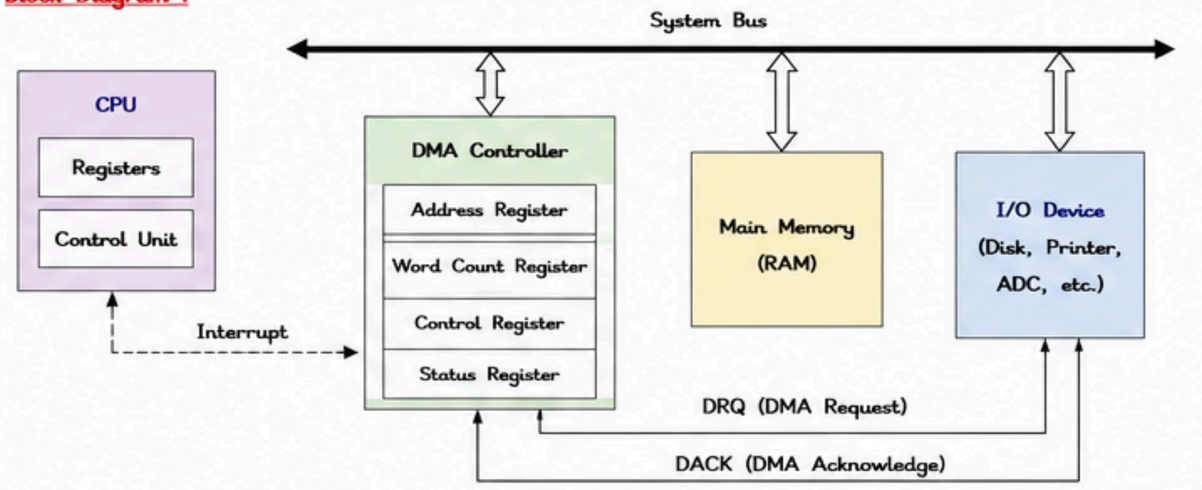
Note : In programmed I/O, data transfer is under direct control of CPU and each data word is moved through CPU.

10 Describe the DMA controller and its operation with a neat diagram.

Ans:

A DMA (Direct Memory Access) controller is a hardware module that transfers data directly between an I/O device and main memory without CPU intervention for each word/byte. It temporarily becomes the bus master and controls the system bus during transfer.

Block Diagram :



Operation (Steps) :

1. **Initialization :** CPU programs DMA controller by writing starting memory address, number of bytes/words to transfer, transfer direction (I/O → Memory or Memory → I/O) and mode.
2. **Request :** I/O device requests DMA transfer by asserting DRQ.
3. **Bus Grant :** DMA controller requests control of system bus from CPU (Bus Request). CPU completes current cycle, releases bus and responds with HLDA (Hold Acknowledge).
4. **Transfer :** DMA controller becomes bus master and transfers data directly between I/O device and memory. It updates address and decrements count after each transfer.
5. **Completion :** When word count becomes zero, DMA controller terminates the transfer, releases bus and interrupts CPU (TC – Terminal Count).
6. **CPU resumes :** CPU receives interrupt and continues execution.

DMA Controller Registers

Register	Function
Address Register (AR)	Holds starting memory address.
Word Count Register (WC)	Holds number of bytes/words to be transferred.
Control Register (CR)	Holds control bits (R/W, I/O device select, mode, start/stop, etc.).
Status Register (SR)	Indicates status (Busy, TC, Error, etc.).

Advantages :

- High speed data transfer.
- CPU is free for other tasks.
- Efficient for large block transfers.
- Reduces CPU overhead.

Applications :

- Disk drives, Printers, Scanners, ADC/DAC.
- Used where large amount of data is transferred.

Note : DMA transfers data in blocks. CPU is involved only at the start (setup) and end (interrupt).